

머신러닝 프로젝트 (RISE) · 2주차

# 프롬프트 기반 개발과 AI 코딩 도구

"코드를 치는 시간보다 무엇을 만들지 설명하는 시간이 길어진다"

- 1단계 바이브코딩 (2~5주) 의 두 번째 시간 — 도구를 손에 익히는 날
- 도구 지형 → 프롬프트 구조화 → 생성·실행·수정 루프 → 생성 코드 검증
- 오늘의 결과물: 내 손으로 돌린 Streamlit 앱 2개 + 프롬프트 로그 1장

# 학습목표와 오늘의 경로

이론 40분 + 실습 80분 — 도구는 무엇이든 좋다, 방법은 하나다

## 학습목표

- AI 코딩 도구의 **종류와 동작 원리**를 이해하고 한계를 설명한다
- 요구사항을 **역할·목표·맥락·제약·출력** 구조의 프롬프트로 쓴다
- 프롬프트 → 코드 → 실행 → 오류 → 수정 **루프**를 혼자 돌린다
- 생성된 코드를 **읽고·테스트하고·기록**한다 (AI 활용 원칙)

| 시간   | 내용                                           | 형태            |
|------|----------------------------------------------|---------------|
| 0:00 | 지난주 복습 · 오늘의 목표                              | 강의 5분         |
| 0:05 | PART 1 도구 지형과 LLM 동작 원리 · LAB 2-1            | 강의 15 + 실습 10 |
| 0:30 | PART 2 프롬프트 구조 · LAB 2-2 (요구사항→프롬프트)         | 강의 10 + 실습 15 |
| 0:55 | PART 3 루프 · LAB 2-3 첫 앱 · 2-4 오류 · 2-5 기능 추가 | 강의 5 + 실습 35  |
| 1:35 | PART 4 검증 · LAB 2-6 설명·테스트·리팩터               | 강의 5 + 실습 10  |
| 1:50 | 정리 · 셀프 체크 · 과제(프롬프트 로그)                     | 10분           |

## 실습 환경

준비물: 1주차와 같은 서버 환경 — <https://ahnbi4.suwon.ac.kr:5300/> 내 계정 로그인 → 터미널 `mkdir -p ~/work/week2 && cd ~/work/week2` → claude. 실습 파일은 `cp -r ~/work/shared/week2/* .`, 데이터 `~/work/shared/data/telco_churn.csv`. 앱은 **내 앱 주소**(`check_env.py` 출력, `.../proxy/8501/`)로 연다.

# 지난주 복습 — 3단계 프로젝트에서 지금 어디인가

1단계는 "빨리 만들어 보여 주기", 2단계는 "잘 맞히기", 3단계는 "둘을 붙이기"

1

1~5주 · 바이브코딩  
웹/앱 프로토타입

2

6~10주 · ML 모델링  
리더보드 경쟁

3

11~15주 · ML 백엔드  
REST API + 프론트 통합

| 주차      | 내용                              | 결과물                           | 평가             |
|---------|---------------------------------|-------------------------------|----------------|
| 2주 (오늘) | 프롬프트 기반 개발 · AI 코딩 도구           | 앱 2개 · 프롬프트 로그                | —              |
| 3주      | 웹/앱 프로토타입 · 화면·기능 연결 · 실행·시연 준비 | 다중 화면 앱 · 시연 가능 상태 · 프로젝트 기획서 | —              |
| 4주      | 개인 프로젝트 개발 (워크숍) · 짝 리뷰         | 기능 명세 · 동작하는 프로토타입            | 중간 피드백         |
| 5주      | 시연·발표·상호 피드백                    | 발표 · AI 활용 기록 · 회고            | <b>1단계 20%</b> |

오늘 만드는 "CSV 요약 앱"과 3주차의 "이탈 위험 체크 앱"은 3단계에서 ML 모델(REST API)을 붙일 **프론트엔드의 뼈대**가 됩니다. 1단계에서 배운 도구 사용법은 2·3단계에서도 매일 씁니다.

# 01

## PART 1

# 바이브코딩과 AI 코딩 도구

무엇을 쓰든 원리는 같다 — 다음 토큰을 확률로 고르는 모델에게 일을 시키는 것

### 이 파트에서

- 1 바이브코딩이란 — 그리고 아닌 것
- 2 도구 지형: 대화형 · IDE 통합 · 에이전트
- 3 LLM은 어떻게 코드를 쓰나 (토큰·컨텍스트·확률)
- 4 LAB 2-1 같은 요청, 다른 프롬프트

# 바이브코딩(vibe coding)이란

자연어로 의도를 말하고, AI가 코드를 쓰고, 사람은 실행 결과를 보며 방향을 잡는 개발 방식 (Karpathy, 2025)

## 바이브코딩에서 사람이 하는 일

- 무엇을 만들지 정한다 — 대상 사용자·문제·기능·제약
- 말로 설명한다 — 프롬프트 (오늘의 핵심 기술)
- 실행해 보고 결과를 판단한다 — 되는가, 원하던 것인가
- 오류를 되돌려 준다 — 메시지·스크린샷·기대와 다른 점
- 책임진다 — 제출물은 내 것, AI 활용은 기록한다

## 바이브코딩이 아닌 것

- 코드를 안 읽어도 되는 것 — 읽지 않으면 고칠 수 없다
- AI가 문제를 정의해 주는 것 — 문제 정의는 사람 몫
- 한 번에 완성되는 것 — 평균 5~15회 왕복이 정상
- 검증을 생략하는 것 — 실행·테스트·읽기 중 하나는 반드시
- 수업계획서 금지 사항: 수정 없이 그대로 제출, 허위 출처

## 평가 기준과 연결

1단계 프로젝트의 평가는 "얼마나 많은 코드를 AI가 썼나"가 아니라 "동작하는가 · 문제가 구체적인가 · 내가 코드를 설명할 수 있는가 · AI 활용을 정직하게 기록했나" 입니다 (5주차 루브릭). 오늘부터 프롬프트 로그를 남깁니다.

# 도구 지형 — 세 가지 유형, 하나의 원리

이 과목의 기본 도구는 ③ Claude Code(서버 터미널). 원리는 셋 다 같으므로 프롬프트도 같다

| 유형         | 대표 도구                                                  | 잘하는 것                             | 주의                                    |
|------------|--------------------------------------------------------|-----------------------------------|---------------------------------------|
| ① 대화형 (웹)  | ChatGPT · Claude · Gemini<br>(Canvas / Artifacts 미리보기) | 기획·설명·오류 해석·짧은 코드·HTML 한 파일 앱     | 내 파일을 모른다 → 코드·오류를 붙여넣어야 함            |
| ② IDE 통합   | VS Code + GitHub Copilot<br>Cursor · Windsurf          | 파일을 보며 수정·자동완성·여러 파일 편집(Agent 모드) | 개인 노트북·개인 계정 필요 → 이 과목에서는 쓰지 않음       |
| ③ 에이전트 CLI | <b>Claude Code</b> (이 과목)<br>Gemini CLI · Codex CLI    | 터미널에서 파일 생성·실행·테스트까지 자율 수행, 긴 작업  | 명령 실행 권한 → 허락 화면을 읽고 승인 (공용 서버·공용 토큰) |

**오늘 실습** ③ Claude Code로 진행 — 내 폴더에서 `claude`, 요청 → 파일 생성 허락(y) → 내가 실행. 개인 계정이 있으면 ①을 오류 해석 보조로

**4주차 개발 워크숍** 내 폴더의 `CLAUDE.md`(프로젝트 규칙 파일)에 구조·규칙을 적어 두면 Claude Code가 매 요청마다 읽는다

## 공용 토큰과 사용 이력

서버의 Claude Code는 **반 전체가 공용 토큰**을 나눠 쓰고, 사용 이력은 **계정별로 기록**된다. "다시 해 봐"를 다섯 번 보내는 사람 하나가 반 전체를 멈출 수 있고, 과도한 사용은 이용 제한 대상. 페이블 모델 금지(/`model` 변경 안 함). 프롬프트를 한 번에 잘 쓰는 습관 (PART 2)이 곧 비용 절감.

# LLM은 어떻게 코드를 쓰나 — 알아야 하는 만큼만

"다음 토큰을 확률적으로 고른다"는 한 문장에서 도구 사용 규칙 다섯 개가 나온다

## 동작 원리 3줄

- **토큰**: 텍스트를 조각(토큰)으로 쪼개어 읽고, 한 조각씩 생성한다
- **컨텍스트 창**: 한 번에 볼 수 있는 분량이 정해져 있다 (수만~수십만 토큰). 창 밖의 내용은 모른다
- **확률 생성**: 학습 데이터에서 자주 본 패턴을 이어 쓴다 → 그럴듯하지만 틀린 코드(환각)가 나올 수 있다

## 그래서 생기는 사용 규칙

- **맥락을 넣어라** — 파일·오류·데이터 구조를 모르면 추측한다
- **작게 쪼개라** — 한 프롬프트에 한 기능. 긴 요청은 앞부분을 잊는다
- **최신 API는 문서로 확인** — 학습 시점 이후 바뀐 함수는 모른다
- **같은 질문도 답이 다르다** — 재현이 필요하면 코드를 저장·버전 관리
- **틀릴 수 있다는 전제로 검증** — 실행·테스트·읽기 (PART 4)

## 실무에서 이렇게 틀린다 ①

흔한 환각 3종: ① 존재하지 않는 함수·인자(`st.chart_bar`) ② 옛 버전 API(`st.cache` → 지금은 `st.cache_data`) ③ 설치 안 된 라이브러리 가정. 셋 다 **실행하면 즉시 드러난다** — 그래서 루프(PART 3)가 중요.

## LAB 2-1 · 같은 요청, 다른 프롬프트

"CSV 요약 함수"를 두 가지 프롬프트로 요청하고 결과를 비교한다 — 10분

### 나쁜 프롬프트 한 줄 요청

csv 분석 코드 짜줘

### 좋은 프롬프트 역할·목표·입출력·제약·예시·형식

너는 pandas에 능숙한 파이썬 개발자다.

[목표] CSV 경로를 받아 숫자 열의 요약표를 돌려주는 함수

[입력] path: str [출력] DataFrame (행=열 이름,  
열=rows, missing, mean, std, min, max)

[제약] pandas만 사용, 숫자 열이 없으면 빈 DataFrame,  
한글 docstring, 함수 이름 summarize\_csv

[예시 데이터] data/telco\_churn.csv (7,043행, 21열,  
TotalCharges는 문자열로 읽힘 - 요약에서 빠져도 됨)

[출력 형식] 코드만, 설명은 주석으로

### 좋은 프롬프트의 결과 practice/week2/summarize\_csv.py

```
def summarize_csv(path: str) -> pd.DataFrame:
    """CSV 숫자 열의 행 수·결측·평균·표준편차·최소·최대"""
    df = pd.read_csv(path)
    num = df.select_dtypes(include="number")
    if num.empty:
        return pd.DataFrame()
    return pd.DataFrame({
        "rows": num.count() + num.isna().sum(),
        "missing": num.isna().sum(),
        "mean": num.mean().round(2), "std": num.std().round(2),
        "min": num.min(), "max": num.max(),
    })
```

```
# python summarize_csv.py data/telco_churn.csv
#
#      rows  missing  mean   std   min   max
# SeniorCitizen  7043      0  0.16  0.37  0.00  1.00
# tenure         7043      0 32.37 24.56  0.00 72.00
# MonthlyCharges 7043      0 64.76 30.09 18.25 118.75
```

해 보기: 나쁜 프롬프트로 받은 코드와 비교 — 함수인가 스크립트인가? 숫자 열이 없을 때는? 열 이름은 내가 원하는 것인가? 차이를 프롬프트 로그에 한 줄로 적는다.

# 02

## PART 2

# 프롬프트의 구조

요구사항을 프롬프트로 구조화한다 — 역할 · 목표 · 맥락 · 제약 · 출력 형식 · 예시

### 이 파트에서

- 1 프롬프트 6요소와 순서
- 2 요구사항 → 사용자 스토리 → 기능 명세 → 프롬프트
- 3 프롬프트 패턴 4종 (생성·수정·설명·계획)
- 4 LAB 2-2 내 서비스의 첫 프롬프트

# 프롬프트 6요소 — 순서대로 쓰면 빠지지 않는다

전부 쓸 필요는 없다. 그러나 빠뜨린 요소는 AI가 "추측"으로 채운다

| 요소      | 무엇을 쓰나                 | 예 (CSV 요약 앱)                                                                |
|---------|------------------------|-----------------------------------------------------------------------------|
| ① 역할    | AI가 어떤 전문가로 답할지        | "Streamlit 앱을 만드는 파이썬 개발자"                                                  |
| ② 목표    | 한 문장으로 만들 것            | "CSV를 올리면 요약 통계와 히스토그램을 보여 주는 앱"                                            |
| ③ 맥락    | 현재 코드·데이터 구조·환경·이미 한 것 | "Python 3.11, streamlit 1.4x, 데이터는 telco_churn.csv 21열, 현재 app_csv.py는 ..." |
| ④ 제약    | 해야 할 것 / 하지 말 것        | "파일 하나, 외부 라이브러리 pandas-streamlit만, 열 이름을 하드코딩하지 말 것"                       |
| ⑤ 출력 형식 | 코드만 / diff만 / 표로 / 단계별 | "전체 파일을 코드 블록 하나로, 바뀐 줄에 주석"                                                |
| ⑥ 예시    | 입력→출력 샘플, 원하는 스타일      | "입력: tenure 열 선택 → 출력: 20구간 막대 그래프 + 평균·결측 캡션"                              |

## 실무에서 이렇게 틀린다 ②

가장 자주 빠지는 것은 ③ 맥락입니다. IDE 통합 도구는 열린 파일을 자동으로 넣어 주지만(@file, #file), 대화형 도구에는 코드·오류·데이터 df.head()를 직접 붙여넣어야 합니다. 맥락 없는 요청 = 추측 코드.

# 요구사항 → 사용자 스토리 → 기능 명세 → 프롬프트

막연한 아이디어를 AI가 실행할 수 있는 단위로 잘게 만드는 4단계 — 4주차 기능 명세서의 뼈대

1

요구사항 (막연)  
"고객 이탈을 관리하고 싶다"

2

사용자 스토리  
"팀원으로서 위험 고객을 빨리 찾기 위해 ..."

3

기능 명세  
입력 5개 → 점수·근거 출력, Must

4

프롬프트  
역할·목표·맥락·제약·형식·예시

## 명세 한 기능 = 한 프롬프트 단위

[사용자 스토리]

고객관리팀원으로서, 위험 고객을 빨리 찾기 위해,  
고객 정보를 입력하면 이탈 위험 점수를 보고 싶다.

[기능 명세 US-1] 우선순위 Must

입력: 계약유형(선택) · 가입개월(0~72) · 월요금(18~120)  
· 인터넷(선택) · 기술지원(예/아니오)

처리: 규칙 기반 점수 0~100 (근거 목록 포함)

출력: 점수 · 수준(높음/보통/낮음) · 근거 · 조회 이력

완료 조건: 폼 제출 → 3초 안에 결과 표시, 빈 입력 방지

## 프롬프트 명세를 그대로 6요소에 태운다

너는 Streamlit 앱을 만드는 파이썬 개발자다.

[목표] 아래 기능 명세 US-1을 구현하는 페이지

[맥락] 프로젝트 구조: app.py(홈), pages/, core.py에  
churn\_score(contract, tenure, monthly, internet,  
support) -> dict(score, level, reasons) 가 이미 있음  
[제약] st.form 사용, core.churn\_score 호출만(로직 재작성 금지),  
session\_state에 조회 이력 저장, 파일 하나  
[출력 형식] pages/2\_이탈\_위험\_체크.py 전체 코드  
[예시] 입력 M2M·3개월·85달러·Fiber·No → 90점 "높음"  
근거 5줄 표시

핵심: **프롬프트는 명세의 번역**이다. 명세가 없으면 프롬프트가 흔들리고, 흔들리는 프롬프트는 매번 다른 코드를 낳는다. 3주차 LAB에서 이 프롬프트의 결과를 실제로 만든다.

# 프롬프트 패턴 4종 — 생성 · 수정 · 설명 · 계획

요청의 종류가 다르면 형식도 다르다. 특히 "수정"은 원본과 바꿀 범위를 함께 준다

| 패턴 | 형식                                          | 예                                                          |
|----|---------------------------------------------|------------------------------------------------------------|
| 생성 | 역할 + 목표 + 제약 + 출력 형식                        | "...앱을 파일 하나로. 코드만."                                       |
| 수정 | <b>현재 코드</b> + 바꿀 것 1가지 + 바꾸지 말 것 + diff 요청 | "아래 코드에서 히스토그램 구간 수를 슬라이더로 조절하게. 다른 부분은 그대로. 바뀐 부분만 보여 줘." |
| 설명 | 코드 + 질문의 초점(흐름·특정 줄·성능·위험)                  | "이 코드에서 <code>st.stop()</code> 이 하는 일과 없었을 때 문제를 3줄로."     |
| 계획 | 목표 + "코드 쓰지 말고" + 단계·파일 목록 요청               | "3화면 앱을 만들려 한다. 코드 말고 파일 구조와 구현 순서, 각 단계의 확인 방법만."         |

**계획 먼저** 기능이 3개 이상이면 "계획" 패턴으로 순서를 받은 뒤 "생성"을 한 단계씩  
**수정은 좁게** "전부 다시 써 줘"는 잘 되던 부분까지 바꾼다. 바꿀 범위를 명시  
**설명은 검증** 설명을 시켜 보면 내가 이해했는지, AI가 헛소리를 하는지 둘 다 드러난다

## 실무에서 이렇게 틀린다 ③

반복 요청이 3번 넘게 실패하면 프롬프트를 고치지 말고 **문제를 더 작게 쪼개거나 맥락을 더 넣어라**. "다시 해 봐"를 5번 반복하는 것은 같은 추측을 5번 시키는 것.

## LAB 2-2 · 내 서비스의 첫 프롬프트 쓰기

아직 주제가 확정되지 않아도 좋다 — 1주차 설문의 아이디어 하나로 스토리 1개 → 명세 1개 → 프롬프트 1개 (15분, 코드 실행 없음)

| 단계        | 쓸 것                                            | 체크                    |
|-----------|------------------------------------------------|-----------------------|
| ① 스토리     | "<누구>로서 <왜> <무엇>을 원한다" 1문장                     | 사용자가 구체적인 한 사람인가      |
| ② 명세      | 입력(위젯·범위) · 처리 · 출력 · 완료 조건                    | 완료 조건을 눈으로 확인할 수 있나   |
| ③ 프롬프트    | 6요소 순서대로. 맥락에 "Streamlit, 파일 하나, 데이터 없음/있음" 명시 | 빠진 요소를 AI가 추측할 여지가 있나 |
| ④ 실행 (선택) | AI에 넣고 결과 코드 저장 → 3주차에 이어서                     | 결과가 명세의 완료 조건을 만족하나   |

### 예시 다른 주제 예 — 형식만 참고

[스토리] 자취생으로서, 식비를 관리하기 위해, 영수증 금액을 입력하면 이번 달 지출 현황을 보고 싶다.

[명세 US-1 Must] 입력: 날짜(기본 오늘)·항목(선택)·금액(정수) / 처리: 세션에 누적 / 출력: 합계·항목별 막대 / 완료 조건: 3건 입력 후 합계가 맞는다

[프롬프트] 너는 Streamlit 개발자다. [목표] US-1 ... [맥락] Python 3.11, streamlit, 데이터 파일 없음, session\_state로 누적 [제약] 파일 하나, pandas·streamlit만 [형식] 코드만 [예시] 3건 입력 → 합계 표시

제출: 오늘 수업 끝나기 전 LMS에 ①~③ 텍스트. 3주차 기획서의 "핵심 기능 3개"가 여기서 나온다.

# 03

## PART 3

# 생성 → 실행 → 수정 루프

바이브코딩의 실체는 한 방이 아니라 짧은 루프의 반복이다

### 이 파트에서

- 1 루프 한 바퀴의 해부 (프롬프트·실행·관찰·되돌려 주기)
- 2 LAB 2-3 첫 Streamlit 앱
- 3 LAB 2-4 오류 메시지를 그대로 주는 법
- 4 LAB 2-5 기능 하나 추가하기

# 루프 한 바퀴 — 프롬프트 · 생성 · 실행 · 관찰 · 되돌려 주기

Streamlit은 저장하면 화면이 바로 바뀌므로 루프가 짧다 — 오늘 도구로 고른 이유

1

프롬프트  
6요소

2

AI 생성  
코드·diff

3

실행  
streamlit run

4

관찰  
동작? 의도?

5

되돌려 주기  
오류·스크린샷·차이

## 되돌려 줄 것 3가지

- **오류 메시지 전체** — 마지막 줄만 말고 Traceback 전부
- **기대 vs 실제** — "히스토그램이 나와야 하는데 빈 화면"
- **바뀐 맥락** — 새로 설치한 것, 바꾼 파일

## 한 바퀴에 하나만

- 오류 하나 → 수정 하나 → 실행. 여러 개를 한꺼번에 던지면 어느 것이 고쳐졌는지 모른다
- 기능 추가는 **되는 상태에서**만 시작 (깨진 채로 추가 금지)
- 되는 버전은 저장/커밋해 두고 다음 바퀴 (3주차 로컬 Git)

## Streamlit 실행 루프

- `streamlit run app.py` → 내 앱 주소(.../proxy/8501/)를 새 탭에
- 파일 저장 → 우상단 "Rerun"/자동 갱신
- 오류는 **브라우저 화면에 Traceback**으로 뜬다 → 복사
- 종료: 터미널 Ctrl+C

## 알아 두기

Streamlit의 규칙 하나: **위젯을 만질 때마다 스크립트 전체가 위에서 아래로 다시 실행**된다. 그래서 "값이 사라진다"는 질문이 가장 많다 — 3주차 `session_state`에서 해결. 오늘은 이 규칙만 기억.

## LAB 2-3 · 첫 Streamlit 앱 — 프롬프트 한 번으로 화면 띄우기

입력 2개(이름·슬라이더) + 버튼 → 메시지. 되면 화면을 캡처해 둔다 (10분)

### 프롬프트

너는 Streamlit 개발자다.  
 [목표] 이름 입력 + 1~10 슬라이더 + "확인" 버튼.  
 버튼을 누르면 "00님, 자신감 N/10 - ..." 성공 메시지  
 [제약] 파일 하나(app\_hello.py), 이름이 비면 경고 표시  
 [출력] 코드만

### 실행

```
streamlit run app_hello.py          # 내 폴더(~/work/week2)에서
# 브라우저 새 탭: 내 앱 주소 .../user/<아이디>/proxy/8501/
# (터미널의 localhost:8501 은 서버 안 주소)
# 종료 Ctrl+C · 파일 저장 후 브라우저 Rerun 버튼
```

### 결과 practice/week2/app\_hello.py

```
import streamlit as st

st.title("첫 번째 바이브코딩 앱")
name = st.text_input("이름을 입력하세요")
level = st.slider("파이썬 자신감 (1~10)", 1, 10, 5)

if st.button("확인"):
    if not name.strip():
        st.warning("이름을 먼저 입력해 주세요.")
    else:
        st.success(f"{name}님, 자신감 {level}/10 - "
                   "오늘 앱을 하나 만들어 보시다!")
```

### 확인할 것

### 어떻게

|                   |                                               |
|-------------------|-----------------------------------------------|
| 빈 이름에서 경고가 뜨나     | 입력 없이 "확인" 클릭 → 노란 경고                         |
| 슬라이더 값이 메시지에 반영되나 | 7로 옮기고 클릭 → "7/10"                            |
| 코드를 읽고 설명할 수 있나   | st.button이 True가 되는 순간은 언제인가? (클릭한 그 재실행 한 번) |

## LAB 2-4 · 오류 메시지를 그대로 되돌려 주기

CSV 업로드 앱 v1은 일부러 깨져 있다 — 브라우저의 Traceback을 복사해 AI에 붙이고 고친다 (10분)

v1 (버그 포함)

practice/week2/app\_csv.py

```
import pandas as pd
import streamlit as st

st.title("CSV 요약 도구")
file = st.file_uploader("CSV 파일을 올려 주세요", type="csv")

if file is not None:
    df = pd.read_csv(file)
    st.write(f"{df.shape[0]:,}행 × {df.shape[1]}열")
    st.dataframe(df.head())
    st.subheader("숫자 열 요약")
    st.dataframe(df.describe().T)
    st.metric("평균 총요금", round(df["총요금"].mean(), 1))
```

브라우저에 뜬 오류

```
KeyError: '총요금'
Traceback:
File "app_csv.py", line 13, in <module>
    st.metric("평균 총요금", round(df["총요금"].mean(), 1))
File ".../pandas/core/frame.py", ... in __getitem__
    indexer = self.columns.get_loc(key)
```

되돌려 주는 프롬프트

아래 Streamlit 앱에서 telco\_churn.csv를 올리면 오류가 난다.  
 [오류] (위 Traceback 전체 붙여넣기)  
 [데이터 열] customerID, gender, ..., MonthlyCharges, TotalCharges(문자열, 공백 섞임), Churn  
 [요청] 존재하지 않는 열을 가정하지 말고, 숫자 열이 있을 때만 요약하도록 고쳐 줘. 바뀐 부분만 diff로.

### 실무에서 이렇게 틀린다 ④

왜 "총요금"인가 — AI가 데이터를 모른 채 그럴듯한 열 이름을 지어낸 전형적 환각. 맥락(열 목록)을 넣으면 재발하지 않는다. 고친 결과는 다음 장 v2의 `select_dtypes + st.stop()` 구조.

## LAB 2-5 · 기능 하나 추가 — "수정" 패턴

v1을 고친 뒤, 열을 골라 히스토그램을 그리는 기능을 한 바퀴에 하나씩 추가 (15분)

### 프롬프트

아래 app\_csv.py에 기능 하나를 추가해 줘.  
 [추가] 숫자 열 중 하나를 selectbox로 고르고, 구간 수  
 슬라이더(5~50, 기본 20)로 히스토그램을 st.bar\_chart로  
 [제약] 기존 요약 부분은 그대로. numpy.histogram 사용.  
 캡션에 평균과 결측 건수  
 [출력] 바뀐 부분만  
 --- (현재 코드 전체 붙여넣기) ---

### 오류 수정본 (LAB 2-4 결과)

```
num_cols = df.select_dtypes(include="number").columns.tolist()
if not num_cols:
    st.warning("숫자 열이 없어 요약할 수 없습니다.")
    st.stop()
st.dataframe(df[num_cols].describe().T)
```

### 추가분

practice/week2/app\_csv\_v2.py

```
col = st.selectbox("히스토그램을 볼 열", num_cols)
bins = st.slider("구간 수", 5, 50, 20)
counts, edges = np.histogram(df[col].dropna(), bins=bins)
hist = pd.DataFrame({"count": counts,
                    index=[f"{edges[i]:.1f}" for i in range(len(counts))])})
st.bar_chart(hist)
st.caption(f"{col}: 평균 {df[col].mean():.2f}, "
          f"결측 {df[col].isna().sum()}건")
```

### 해 보기

### 프롬프트 힌트

|              |                                |
|--------------|--------------------------------|
| x축 라벨이 너무 많다 | "구간 라벨을 5개마다 하나만"              |
| 다운로드 버튼      | "요약표를 CSV로 st.download_button" |
| 열 2개 산점도     | "st.scatter_chart로 x-y 선택"     |

한 바퀴마다 되는 버전을 다른 이름으로 저장(v1, v2 ...)하거나 커밋. "잘 되던 게 갑자기 안 된다"는 말의 90%는 되돌아갈 버전이 없어서 생긴다.

# 04

PART 4

## 생성 코드 검증

읽는다 · 돌려 본다 · 테스트를 시킨다 · 기록한다 — 네 가지 중 최소 둘

이 파트에서

- 1 검증 사다리 4단
- 2 "설명해 줘"가 최고의 코드 리뷰인 이유
- 3 LAB 2-6 설명 · 테스트 · 리팩터 요청
- 4 AI 활용 기록 (프롬프트 로그)

# 검증 사다리 — 비용은 올라가고 확신도 올라간다

프로토타입 단계는 1·2단이 기본, 내 프로젝트의 핵심 함수는 3단까지

| 단 | 방법       | 무엇을 잡나                           | 비용  |
|---|----------|----------------------------------|-----|
| 1 | 실행해 본다   | 문법 오류·없는 함수·설치 안 된 패키지 — 환각의 대부분 | 초   |
| 2 | 읽고 설명시킨다 | 의도와 다른 로직, 하드코딩된 값, 빠진 예외 처리     | 분   |
| 3 | 테스트를 시킨다 | 경계값·빈 입력·타입 오류 — 재수정 후에도 남는 회귀   | 10분 |
| 4 | 사람이 리뷰한다 | 보안(비밀키 노출)·성능·구조 — 3단계 통합 전      | 시간  |

**"설명해 줘"의 효과** 설명이 코드와 어긋나면 둘 중 하나가 틀렸다. 내가 못 따라가면 코드가 너무 복잡하다는 뜻 — 단순화 요청

**테스트는 순수 함수부터** 화면(Streamlit)은 테스트가 어렵다. 로직을 함수(core.py)로 빼면 테스트도, 3단계 API 전환도 쉽다

## 실무에서 이렇게 틀린다 ⑤

"코드를 안 읽어도 되는 게 바이브코딩 아닌가요?" — 프로토타입 시연까지는 가능하다. 그러나 5주차 Q&A에서 **본인이 데이터 흐름을 설명**해야 하고(루브릭 20점), 3단계에서 남의 코드에 API를 붙여야 한다. 읽는 습관은 오늘부터.

## LAB 2-6 · 설명 · 테스트 · 리팩터를 시킨다

LAB 2-1의 `summarize_csv`에 대해 세 가지 요청을 순서대로 (10분)

### 세 프롬프트

- ① 설명: 아래 함수가 하는 일을 3줄로, 그리고 입력이 비었거나 숫자 열이 없을 때 무슨 일이 생기는지.
- ② 테스트: `pytest`로 테스트 2개 - (a) 문자열 열은 빠지고 결측 수가 맞는지 (b) 숫자 열이 없으면 빈 `DataFrame`인지. `tmp_path`로 임시 `CSV`를 만들어 쓸 것.
- ③ 리팩터: `rows` 계산이 어색하다(`count + isna`). 더 단순한 표현으로 바꾸되 결과는 같아야 한다.

### ② 결과 test\_summarize\_csv.py → pytest -q: 2 passed

```
def test_numeric_columns_only(tmp_path):
    p = tmp_path / "t.csv"
    pd.DataFrame({"a": [1, 2, None], "b": ["x", "y", "z"]})
        .to_csv(p, index=False)
    out = summarize_csv(str(p))
    assert list(out.index) == ["a"]
    assert out.loc["a", "missing"] == 1

def test_no_numeric_returns_empty(tmp_path):
    p = tmp_path / "t.csv"
    pd.DataFrame({"b": ["x", "y"]}).to_csv(p, index=False)
    assert summarize_csv(str(p)).empty
```

### 요청

### 보는 것

#### ① 설명

"숫자 열이 없으면 빈 `DataFrame`" — 코드와 일치하나? 설명에 없는 동작(예: 결측 열 처리)이 코드에 있나?

#### ② 테스트

테스트가 **통과하는지**보다, 테스트가 **내 명세**를 검사하는지. 명세에 없는 것을 검사하면 지운다

#### ③ 리팩터

`len(num)` 한 줄이면 된다. 바뀐 뒤 테스트 2개가 여전히 통과하는지 = 회귀 확인

리팩터 후 `pytest -q`를 다시 돌리는 것까지가 한 바퀴. 테스트가 있으면 AI의 수정을 겁내지 않아도 된다 — 4주차에 AI가 내 코드를 크게 고칠 때 특히.

# AI 활용 기록 — 프롬프트 로그

수업계획서 「학습자 AI 활용 원칙」: 활용 사실 명확히 표기 · 수정 없이 그대로 제출 금지 · 허위 출처 금지

| # | 도구          | 목적          | 프롬프트 요약             | 어떻게 검증·수정했나                             | 채택      |
|---|-------------|-------------|---------------------|-----------------------------------------|---------|
| 1 | Claude Code | CSV 요약 함수   | 역할·입출력·예외 명시        | 결측 3건 파일로 직접 실행, rows 계산 단순화            | 수정 후 채택 |
| 2 | ChatGPT     | KeyError 해석 | Traceback + 열 목록    | 제안대로 <code>select_dtypes</code> , 실행 확인 | 채택      |
| 3 | Claude Code | 히스토그램 추가    | 수정 패턴, 범위 명시        | x축 라벨 겹침 → 직접 5개마다 표시로 수정               | 수정 후 채택 |
| 4 | Claude      | 테스트 생성      | pytest 2개, tmp_path | 명세 밖 테스트 1개 삭제                          | 일부 채택   |

**왜 남기나** ① 정직성 증빙 ② 다음 주의 내가 같은 삽질을 반복하지 않게 ③ 5주차 시연의 "AI 활용·배운 점" 30초 재료

**무엇을 남기나** 전부가 아니라 **결정이 바뀐 프롬프트** 10개 이내. "다시 해 봐"류는 제외

**어디에** templates/03\_AI\_활용\_기록\_템플릿.md — 내 폴더에 AI\_LOG.md로 두고 함께 커밋

## 채점 관점

"AI가 틀렸던 사례"를 적는 사람이 루브릭 **AI 활용 기록 15점** 중 상위. 틀린 사례가 없다는 것은 검증을 안 했다는 뜻으로 읽힌다.

# 05

PART 5

## 정리와 과제

오늘 배운 것을 한 표로, 그리고 다음 주까지 할 것

이 파트에서

- 1 한 장 요약
- 2 셀프 체크 5문항
- 3 과제: 프롬프트 로그 + 앱 1개
- 4 3주차 예고 — 다중 화면 프로토타입과 배포

# 한 장 요약

도구는 바뀌어도 이 표는 바뀌지 않는다

| 주제        | 핵심                                         | 오늘 실습에서                         |
|-----------|--------------------------------------------|---------------------------------|
| 도구 지형     | 대화형 · IDE 통합 · 에이전트 — 원리는 같다(확률 생성·컨텍스트 창) | Claude Code (서버 터미널)            |
| 프롬프트 6요소  | 역할 · 목표 · 맥락 · 제약 · 출력 형식 · 예시. 빠진 것은 추측된다 | LAB 2-1 나쁜 vs 좋은, LAB 2-2 내 서비스 |
| 명세 → 프롬프트 | 스토리 → 기능 명세(입력·처리·출력·완료 조건) → 프롬프트         | LAB 2-2                         |
| 루프        | 한 바퀴에 하나 · 오류는 전체 붙여넣기 · 되는 버전 저장          | LAB 2-3 첫 앱, 2-4 오류, 2-5 추가     |
| 검증 사다리    | 실행 → 설명 → 테스트 → 리뷰. 로직은 함수로 분리             | LAB 2-6 pytest 2개               |
| 기록        | 프롬프트 로그 10개 이내, 틀린 사례 포함                   | 템플릿 03                          |

## 셀프 체크 — 5문항

옆 사람과 1분씩. 답은 오늘 슬라이드 안에 있다

- **Q1.** "csv 분석 코드 짜줘"에 빠진 프롬프트 요소를 셋 이상 말하라.
- **Q2.** AI가 `df["총요금"]`을 쓴 이유는 무엇이고, 어떤 맥락을 주면 재발하지 않나?
- **Q3.** Streamlit에서 버튼을 누르면 스크립트는 몇 번, 어디서부터 실행되나?
- **Q4.** "전부 다시 써 줘" 대신 수정 패턴에서 반드시 함께 주는 두 가지는?
- **Q5.** 검증 사다리 1단(실행)이 잡는 오류와 못 잡는 오류를 하나씩 들라.

정답 요지 — Q1 역할·맥락(데이터 구조)·제약·출력 형식·예시 / Q2 데이터를 모른 채 그럴듯한 이름을 생성(환각), 열 목록·`df.head()`를 넣는다 / Q3 한 번, 맨 위부터 전체 / Q4 현재 코드 + 바꾸지 말 범위(그리고 diff 요청) / Q5 잡음: 없는 함수·문법 · 못 잡음: 의도와 다른 로직·경계값

# 과제 — 다음 주 수업 전까지 (개인)

LMS 제출. 3주차 첫 10분에 몇 명의 앱을 화면에 띄운다

| # | 과제                              | 제출물                                           | 기준                               |
|---|---------------------------------|-----------------------------------------------|----------------------------------|
| 1 | LAB 2-5 앱에 "해 보기" 중 하나 이상 추가    | app_csv_v3.py + 실행 화면 캡처                      | 실행되고, 추가 기능이 동작                  |
| 2 | LAB 2-2에서 쓴 프롬프트로 실제 코드 생성 → 실행 | 코드 파일 + 캡처 + 완료 조건 충족 여부 한 줄                  | 완료 조건 대비 결과를 정직하게                |
| 3 | 프롬프트 로그 (과제 1-2 과정)             | 템플릿 03, 5~10행                                 | "틀렸던 사례" 1개 이상                   |
| 4 | (준비) 내 앱을 폰으로 열어 보기             | 폰 브라우저로 서버에 <b>내 계정 로그인</b> → 내 앱 주소를 연 화면 캡처 | 5주차 시연 준비의 여행연습 (앱은 내 계정에서만 열린다) |

## 다음 주

3주차 예고: **웹/앱 프로토타입 구현** — 화면 설계(와이어프레임→프롬프트) · 레이아웃·상태(session\_state)·다중 페이지 · 실행·시연 준비(앱 전용 터미널 → 내 앱 주소) · 로컬 Git 스냅샷. 수업 후 **내 주제 확정**(기획서 1장). 수업 전에 서비스 아이디어 2개를 생각해 오세요.

- 참고: Karpathy, "vibe coding" (X, 2025.2) · Claude Code Docs "Common workflows" · Streamlit Docs "Get started" · Anthropic "Prompt engineering overview"